00

The toolkit

How the course is used, and the prompts every other chapter assumes you have

This course teaches mental models, and it hands you prompts for investigating a project with an assistant. You can read all of it with no project at all. When you do have one — yours, a client's, something you inherited — the prompts are what you run, and the checks are how you find out whether what came back is true. This chapter is the ten minutes of setup the rest assumes: what an assistant needs in order to investigate, how to read the lists it returns, and three things you may not know how to do that your assistant can do for you.

PART

The model

Two kinds of tool

Every chapter closes with two kinds of thing to run. An **ask** is a prompt that produces a list — every table with its policies, every endpoint with who can call it, every place a secret is used. You judge the list by its shape, without understanding its contents. A **check** is an observation that settles whether the list was right — a request you send, a screen you look at, a row you read. Asks are cheap and can be wrong; checks are slower and can't be argued with. The chapters use both, and chapter 12 is about deciding how much checking a change deserves.

An assistant that can read the project

The asks assume an assistant that works inside your files — Claude Code, Cursor, Codex, Windsurf, and their equivalents. A chat window works too, with the relevant files pasted in first, and the list it returns covers only what you pasted. The first prompt to run on any project is the one that tells you what you're standing on:

> What's my stack? Name the framework, the language, the database, the login system, the host, and where the code's history lives. One line each. Say unknown where you can't tell.

That answer is the vocabulary for everything after — advice transfers within a stack far better than across, and knowing the names lets you ask about your actual setup rather than the general case.

Reading a list for its shape

A vague question has no failing answer. "Are there any authorization gaps?" returns "this looks good" reliably, because nothing in the question forces a finding to surface. The asks throughout the course are built from six patterns, and once you can see them you can write your own:

1. **A list with a completeness criterion** — “every table, all four operations; flag any with fewer than four.” You can count rows and flags without knowing what the operations mean.
2. **State the source** — “for each write, does the user’s identity come from the session or the request body?” A forced choice between a right answer and a wrong one.
3. **Say none if absent** — the instruction that turns a silent omission into a visible word. Without it, a missing thing produces nothing; with it, a missing thing produces “none,” which you can read.
4. **Walk the failure path** — “step by step, what stops this bad request, and does anything actually?” The walk has to reach a definite ending: blocked at a named step, or not.
5. **List what this depends on** — surfaces the neighbour that the change quietly leans on.
6. **Plant a control** — before trusting a list, seed one finding you already know about and see whether the list surfaces it. A list that misses the planted item wasn’t complete. A list that has never had this done to it has never been shown able to find anything, and that includes every list in this course until you’ve tried it once.

The anti-patterns are yes/no questions, “any problems?”, and anything the assistant can satisfy with a summary. And where you ask matters as much as how: the session that wrote the code is the session most likely to defend it, so audits run in a fresh session with the code read cold.

Three things your assistant can do for you

Some checks need a capability you may not have, and none of them needs learning. The move, every time, is to hand the setup to the assistant and ask for the exact thing to run. Three come up throughout the course.

A copy of the database you can break. Several checks say “in a test copy”: insert a row your product forbids, delete a test account, make a write fail halfway. The copy is what makes that safe. Your platform has a way to get one — a branch, a second project, a local database seeded from a dump — and the assistant knows which.

Re-sending a request from the browser. Chapters 1, 3, 4 and 6 have you open the browser’s developer tools, find the request a form sent, and send it again with something changed.

It’s a two-minute skill once shown, and it’s the single most useful thing a non-programmer can learn to do to their own app, since it’s how you find out what the server accepts rather than what the form allows.

Building the client the way production does. Chapter 7’s direct test searches the built client for secrets. The build is a command; the assistant knows which one and where the output lands.

The going-deeper section has the three prompts. Run whichever a chapter’s check needs, when it needs it.

If you have no project yet

The models read the same without one, and the prompts wait. When you want something to run them on, any project you can point an assistant at will do — one you’ve built, one you’ve inherited, a small open-source app you’ve cloned — and the first prompt above tells you what it’s made of. A sandbox project is in preparation for this course: a small app with known findings planted in it, so the asks can be tried where the right answer is known before they’re pointed at something real. Until it exists, your own project is the practice ground, and *plant a control* is how you check an ask before trusting it there.

What a finding is

A finding is a row on a list, or a check that came back wrong. It isn’t an emergency and it isn’t a verdict — it’s the start of a conversation with the assistant, in which you ask what the fix is, what it touches, and how you’d know it worked. Chapter 12 decides how much evidence that conversation should end with, by what the change touches.

PART

Going deeper

These are the capability prompts the other chapters' checks assume, plus one for the rules file. Each comes with a note on what a good response looks like.

D1 · A copy I can break

BUILD A TOY

I want a copy of this project's database that I can damage without affecting anything real — a branch, a second project, or a local database seeded from the real one, whichever fits this stack. Set it up with me step by step, tell me how to point the app at it and back, and finish by showing me one harmless write against the copy so I can see it's separate.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response ends with two things you can see: the copy's name or address, and the app running against it. If the assistant proposes working against the real database "carefully," decline — the copy is the point.

D2 · Re-send a request

BUILD A TOY

Show me how to re-send one of my own app's requests from the browser. Walk me through it: open the developer tools, find the network tab, submit [a form in my app], find the request it sent, copy it as a fetch call, and send it again from the console with one value changed to something the form wouldn't allow. Tell me exactly what to click and what to paste, and help me read what comes back.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response is a sequence you can follow with the browser open, ending with a response you can interpret — accepted or rejected. Once you've done it once you can do every check in the course that says "send the request directly."

D3 · Build the client

BUILD A TOY

Build this project's client the way production builds it, and tell me where the output is. Then search the output for the first eight characters of each secret in my environment file, and show me the exact command you used so I can run it again after changes.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response names the build command, the output folder, and the search, and reports each secret as found or not found. Any "found" is chapter 7's subject.

D4 · Find the rules file

GROUNDED EXPLANATION

Which file does my assistant read for standing instructions in this project, and does one exist yet? Name it for my tool — `CLAUDE.md` for Claude Code, `AGENTS.md` for Codex, `.cursor/rules` for Cursor, `.windsurfrules` for Windsurf, `.github/copilot-instructions.md` for Copilot — and create an empty one with a one-line heading if it's missing.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response names one file and confirms it exists. Chapters 6 and 11 give you rules to put in it.

WHERE THIS CONNECTS

Chapter 1 · Client and server is where the re-send skill first matters, and the chapter to read next. **Chapter 12 · Verification** turns the finding into a decision about evidence. **Chapter 11 · The loop** is where the rules file earns its keep.